

Part 01 : Theoretical Questions

Q1 : Consider the following class:

```
public class BankAccount
{
    public string Owner;
    public double Balance;

    public void Withdraw(double amount)
    {
        Balance -= amount;
    }
}
```

- a) Identify at least two problems with this design from an encapsulation perspective.
- b) Describe how you would fix this class to follow proper encapsulation principles. You do not need to write the full code.
- c) Explain why exposing fields directly (as public) is considered a bad practice in OOP.

Q02 : What is the difference between a field and a property in C#? Can a property contain logic? Give an example of a read-only property that returns a calculated value.

Q3 : Look at the following code and answer the questions below:

```
public class StudentRegister
{
    private string[] names = new string[5];

    public string this[int index]
    {
        get { return names[index]; }
        set { names[index] = value; }
    }
}
```

- a)** What is `this[int index]` called? Explain its purpose.
 - b)** What happens if someone writes `register[10] = "Ali";`? How would you make the indexer safer?
 - c)** Can a class have more than one indexer? If yes, give an example of when that would be useful.
-

Q4 : Consider the following code and answer the questions below:

```
public class Order
{
    public static int TotalOrders = 0;
    public string Item;

    public Order(string item)
    {
        Item = item;
        TotalOrders++;
    }
}
```

- a)** What does the `static` keyword mean on `TotalOrders`? How is it different from the `Item` field?
- b)** Can a static method inside `Order` access the `Item` field directly? Why or why not?

Part 02 : Practical (Extending the Movie Ticket Booking System)

In Assignment 01, you built a basic Movie Ticket Booking System with a Ticket class, a SeatLocation struct, and a TicketType enum. Now you will improve and extend that system using encapsulation, properties, indexers, and static members.

What you need to build :

1. Refactor the Ticket class to use proper encapsulation:
 - a. Create public properties for each field with the following validation rules:
 - MovieName : cannot be null or empty. If an invalid value is set, keep the previous value.
 - Type : use the TicketType enum from Assignment 01 (no special validation needed).
 - Seat : use the SeatLocation struct from Assignment 01 (no special validation needed).
 - Price : must be greater than 0. If an invalid value is set, keep the previous value.
 - b. Add property `PriceAfterTax` that returns the price with 14% tax included (calculated, not stored).
-

- 2.** Add a static field and a static method to the Ticket class:
- a.** Add a 'ticketCounter' field that starts at 0.
 - b.** Add a 'TicketId' property. Each ticket gets a unique ID automatically when created (increment 'ticketCounter' in the constructor and assign it to the ID).
 - c.** Add a 'GetTotalTicketsSold()' method that returns the current value of 'ticketCounter'.
-

- 3.** Create a 'Cinema' class that holds up to 20 tickets using a private array. Add the following:
- a.** Allow User To get and set tickets by index if the index is out of range, the getter returns null and the setter does nothing.
 - b.** Allow User To Get Movie By movieName that returns the first ticket found matching the given movie name, or null if not found.
 - c.** A method 'AddTicket(Ticket t)' that adds a ticket to the first available (null) slot. Returns true if added, false if the cinema is full.
-

- 4.** Create a static utility class called 'BookingHelper' with the following static methods:
- a.** 'double CalcGroupDiscount(int numberOfTickets, double pricePerTicket)' That returns total price with a 10% discount if the group has 5 or more tickets, otherwise returns the full total.
 - b.** 'string GenerateBookingReference()' That returns a unique string each time it is called (e.g., "BK-1", "BK-2", "BK-3", ...). Use a private static counter internally.

5. In your `Main` method, build a Console Application that does the following:
- Ask the user to enter data for 3 tickets (movie name, ticket type, seat row, seat number, price). Create each Ticket and add it to the Cinema .
 - Print all 3 tickets (access by index 0, 1, 2) showing: TicketId, MovieName, Type, Seat, Price, and PriceAfterTax.
 - Ask the user for a movie name and search for it. Print the result or a "not found" message.
 - Print the total tickets sold using the method.
 - Generate and print 2 booking references .
 - Calculate and print the group discount for a group of 5 tickets at 80 EGP each of them.

Expected Output (Example) :

```
===== Ticket Booking =====

Enter data for Ticket 1:
Movie Name: Inception
Ticket Type (0=Standard, 1=VIP, 2=IMAX): 1
Seat Row (A-Z): B
Seat Number: 5
Price: 120

Enter data for Ticket 2:
Movie Name: Avengers
Ticket Type (0=Standard, 1=VIP, 2=IMAX): 0
Seat Row (A-Z): A
Seat Number: 3
Price: 80

Enter data for Ticket 3:
Movie Name: Dune
Ticket Type (0=Standard, 1=VIP, 2=IMAX): 2
Seat Row (A-Z): C
Seat Number: 10
Price: 200

===== All Tickets =====

Ticket #1 | Inception | VIP | Seat: B-5 | Price: 120 EGP | After Tax: 136.8 EGP
Ticket #2 | Avengers | Standard | Seat: A-3 | Price: 80 EGP | After Tax: 91.2 EGP
Ticket #3 | Dune | IMAX | Seat: C-10 | Price: 200 EGP | After Tax: 228 EGP

===== Search by Movie =====
Enter movie name to search: Avengers
Found: Ticket #2 | Avengers | Standard | Seat: A-3 | Price: 80 EGP

===== Statistics =====
Total Tickets Sold: 3

Booking Reference 1: BK-1
Booking Reference 2: BK-2

Group Discount (5 tickets x 80 EGP): 360 EGP (10% off applied)
```